Electronics and
Computer Science

UNIVERSITY OF
Southampton

3. RISC-V Instructions

RISC vs CISC

- RISC concept dates to about 1980, David Patterson, UC Berkeley
  - There were RISC-like processors before then.
  - Patterson decided not to use microcode in a design
- "Reduced Instruction Set Computer"
  - "Reduced" refers to the work of a single instruction, not to the number of instructions
  - "Relegate Interesting Stuff to the Compiler"
- Simple instructions, regular pattern
- CISC processors now have RISC features

Instruction Set Design

- There's no point in creating hardware that a compiler can't use
- RISC-V was designed with 3 principles:
  1. Simplicity favours regularity
  2. Smaller is faster
  3. Good design demands good compromises

C to Assembly to Machine Code

```
int add (int b, int c) {
   int a;
   a = b+c;
   return a;
   }
```

Let's compile this for different Instruction Sets

```
$ gcc -c add.c
$ objdump -d add.o
```

x86-64 Assembly

```
 0:   f3 0f 1e fa             endbr64
 4:   55                      push   %rbp
 5:   48 89 e5                mov    %rsp,%rbp
 8:   89 7d ec                mov    %edi,-0x14(%rbp)
 b:   89 75 e8                mov    %esi,-0x18(%rbp)
 e:   8b 55 ec                mov    -0x14(%rbp),%edx
11:   8b 45 e8                mov    -0x18(%rbp),%eax
14:   01 d0                   add    %edx,%eax
16:   89 45 fc                mov    %eax,-0x4(%rbp)
19:   8b 45 fc                mov    -0x4(%rbp),%eax
1c:   5d                      pop    %rbp
1d:   c3                      ret
```

Function header and footer

eax, edx (and ebx, ecx) 32 bit registers

Machine Code

Assembly language

Arm 64 Assembly

```
 0: d10043ff     sub sp, sp, #0x10
 4: b9000fe0     str w0, [sp, #0xc]
 8: b9000be1     str w1, [sp, #0x8]
 c: b9400fe8     ldr w8, [sp, #0xc]
10: b9400be9     ldr w9, [sp, #0x8]
14: 0b090108     add w8, w8, w9
18: b90007e8     str w8, [sp, #0x4]
1c: b94007e0     ldr w0, [sp, #0x4]
20: 910043ff     add sp, sp, #0x10
24: d65f03c0     ret
```

RISC-V 64 bit Assembly

```
 0:    fd010113                addi    x2,x2,-48
 4:    02113423                sd      x1,40(x2)
 8:    02813023                sd      x8,32(x2)
 c:    03010413                addi    x8,x2,48
10:    00050793                addi    x15,x10,0
14:    00058713                addi    x14,x11,0
18:    fcf42e23                sw      x15,-36(x8)
1c:    00070793                addi    x15,x14,0
20:    fcf42c23                sw      x15,-40(x8)
24:    fdc42783                lw      x15,-36(x8)
28:    00078713                addi    x14,x15,0
2c:    fd842783                lw      x15,-40(x8)
30:    00f707bb                addw    x15,x14,x15
34:    fef42623                sw      x15,-20(x8)
38:    fec42783                lw      x15,-20(x8)
3c:    00078513                addi    x10,x15,0
40:    02813083                ld      x1,40(x2)
44:    02013403                ld      x8,32(x2)
48:    03010113                addi    x2,x2,48
4c:    00008067                jalr    x0,0(x1)
```

RISC-V 32 bit Assembly

```
  0:   fd010113          addi    x2,x2,-48
  4:   02812623          sw      x8,44(x2)
  8:   03010413          addi    x8,x2,48
  c:   fca42e23          sw      x10,-36(x8)
 10:   fcb42c23          sw      x11,-40(x8)
 14:   fdc42703          lw      x14,-36(x8)
 18:   fd842783          lw      x15,-40(x8)
 1c:   00f707b3          add     x15,x14,x15
 20:   fef42623          sw      x15,-20(x8)
 24:   fec42783          lw      x15,-20(x8)
 28:   00078513          addi    x10,x15,0
 2c:   02c12403          lw      x8,44(x2)
 30:   03010113          addi    x2,x2,48
 34:   00008067          jalr    x0,0(x1)
```

RISC-V architecture summary

- RISC – 5th generation reduced instruction set computer, developed by University of California, Berkeley in 2010.
- Open source instruction set architecture (ISA)

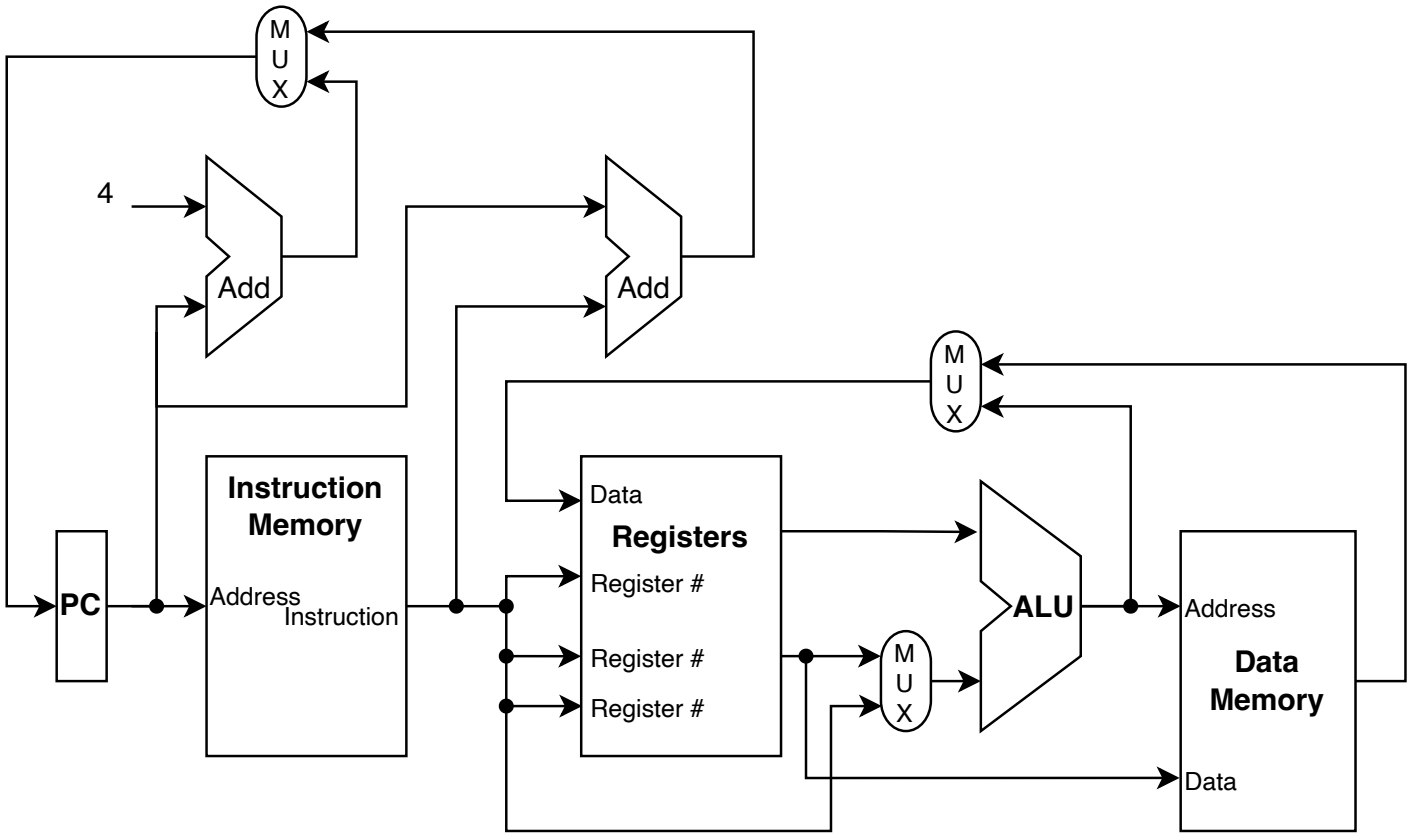
- A range of versions, architecture is not specified
  - 32-bit vs 64-bit
  - Single cycle CPU vs pipelined CPU
  - Extensible instruction set

Instruction Sets and Extensions

RV32I	Base 32-bit integer instruction set, with 32 registers
RV32E	Base 32-bit integer instruction set, with 16 registers
RV64I	Base 64-bit integer instruction set, with 64-bit registers
M	Adds integer multiply and divide
A	Adds atomic instructions for concurrent processing
F	Adds 32-bit floating point arithmetic
C	Defines 16-bit compressed versions of common instructions
D, Q, L, V, B, T, RV128I	Further floating point and other future extensions

The IBEX processor (introductory lab) supports RV32I plus the M and C extensions. (rv32imc)
We will focus on the RV32I set here.

Simple RISC-V Architecture



Control signals not shown

Core Instruction Formats

	31 25	24 20	19 15	14 12	11 7	6 0
R	funct7	rs2	rs1	funct3	rd	opcode
I	imm[11:0]		rs1	funct3	rd	opcode
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B	imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]				rd	opcode
J	imm[20|10:1|11|19:12]				rd	opcode

Regularity – Note how imm bits are shuffled to maintain regularity

Add Instruction

- R-type instruction
  - `add rd, rs1, rs2`
- Previous example
  - `add x15, x14, x15`
- In machine code
  - `00f707b3`
  - `0000_0000_1111_0111_0000_0111_1011_0011`
- Regroup
  - `0000000_01111_01110_000_01111_0110011`

funct7	000000	funct3	000
rs2	01111	rd	01111
rs1	01110	opcode	0110011

Sub Instruction

```
a = b - c;
```

- Becomes

```
40f707b3      sub x15, x14, x15
```

- Exactly the same, except

```
funct7 0100000
```

- All R-type instructions have the same opcode (`0110011`), but differ in funct3 and funct7

```
sll, slt, sltu, xor, srl, sra, or, and
```

addi Instruction

```
a = b + 14;
```

- Becomes

```
00e78793      addi x15, x15, 14
```

- This is an I-type instruction
  - `0000_0000_1110_0111_1000_0111_1001_0011`
- Regroup – `imm(11:0)` is now the most significant 12 bits
  - `000000001110_01111_000_01111_0010011`

imm(11:0)	00000001110	funct3	000
		rd	01111
rs1	01111	opcode	0010011

- Different opcode, same funct3

Exercise

- Modify the add function for different operators (register and immediate)

```
int add (int b, int c) {
  int a;
  a = b + c;
  return a;
  }
```

- Compile for RV32I on WSL

```
riscv32-unknown-elf-gcc -c -march=rv32i add.c
```

- Dump object file as text

```
riscv32-unknown-elf-objdump -M no-aliases -M numeric -d
add.o
```

Exercise

- Find the relevant instruction. Identify the fields.
- What do you see if you omit `-march=rv32i` ?
- What do you see if you omit `-M no-aliases` and/or `-M numeric` ?